Chapter 3: Bracketing

Algorithms for Optimization, 2nd Edition (2026)

Kochenderfer & Wheeler

Lecture Slides

Outline

What Is Bracketing and Why Does It Matter? I

Optimization problems ask: "Where does this function achieve its smallest value?" A naive approach would be to evaluate the function at thousands of random locations, but this is wasteful. **Bracketing** gives us a smarter strategy: rather than searching the entire real line, we first identify a *region* (an interval) that is guaranteed to contain the minimum, and then progressively *shrink* that region until the minimum is pinpointed to whatever precision we desire.

Definition: Bracketing

Bracketing is the process of identifying an interval $[a, c]$ — read "the closed interval from a (the left endpoint) to c (the right endpoint), inclusive" — within which a local minimum of the function f is guaranteed to lie, and then progressively narrowing that interval. The notation $[a, c]$ means all real numbers x satisfying $a \leq x \leq c$.

Bracketing applies exclusively to **univariate functions** — functions that take a single real-valued input x and return a single real-valued output $f(x)$. The letter x represents the variable we are optimising over, and $f(x)$ represents the "cost" or "objective" we want to minimise. If we know the minimum is

What Is Bracketing and Why Does It Matter? II

inside $[a, c]$, every evaluation of f at an interior point tells us something about the shape of f and lets us *eliminate* part of the interval. Repeating this procedure drives the interval length toward zero. Derivative information — the slope $f'(x)$ of the function at a point, read "f prime of x" — can accelerate the search when it is available, but several of the methods in this chapter work with **function values alone**. They treat f as a "black box" that can only be queried at chosen points, with no access to slope information. This is particularly valuable when f comes from a simulation, a physical experiment, or any other process where computing the derivative is impractical.

The methods in this chapter form the critical foundation for **line search** in multi-variable optimisation (covered in later chapters): when we minimise a function of many variables by moving along one direction at a time, each move reduces to a one-dimensional problem of exactly the type studied here.

Key Insight

We do not need to evaluate f everywhere. A handful of cleverly placed evaluations, combined with the right structural assumption about f (such as *unimodality*, defined on the next slide), is enough to locate the minimum accurately. The art of bracketing is choosing *where* to evaluate in order to learn the most from each query.

3.1 Unimodality: The Key Assumption I

Before we can safely discard part of a bracketing interval, we need a structural guarantee that discarding it will not throw away the true minimum. The assumption that provides this guarantee is called *unimodality*.

Definition: Unimodal Function

A function f is called **unimodal** (from Latin: *unus* = one, *modus* = mode or peak) if it has *exactly one* local minimum, and the function strictly decreases on one side of that minimum and strictly increases on the other. More precisely, there exists a unique point x^* (read "x star"; the superscript $*$ is a conventional symbol meaning "optimal") such that:

$$f(x_1) > f(x_2) \text{ for all } x_1 < x_2 \leq x^* \qquad \text{and} \qquad f(x_1) < f(x_2) \text{ for all } x^* \leq x_1 < x_2.$$

In plain language: the function consistently decreases as we approach x^* from the left, and consistently increases as we move away from x^* to the right. There are no other "dips" or "bumps".

3.1 Unimodality: The Key Assumption II

Notation for Unimodal Definition

x^* (x star): the unique global minimiser — the point where f achieves its smallest value.
x_1, x_2 (x sub 1, x sub 2): two arbitrary points in the domain used to express the monotonicity condition. The subscripts 1 and 2 label two distinct points, not specific values.
$f(x_1) > f(x_2)$: the function value at x_1 is strictly greater than at x_2, meaning x_1 is "higher up" than x_2. The symbol $\leq$ means "less than or equal to" and $<$ means "strictly less than".

A helpful mental image: think of a smooth valley — like the letter "U" — with a unique lowest point at the bottom. A function such as $f(x) = (x-2)^2$ is unimodal with minimum at $x^* = 2$: moving away from $x = 2$ in either direction causes f to increase.

3.1 Unimodality: The Key Assumption III

Definition: Multimodal Function

A function is called **multimodal** (from Latin: *multi* = many) if it has more than one local minimum — multiple "valleys" or "dips". A classic example is $f(x) = \sin(x)$ (read "sine of x", a trigonometric function that oscillates between -1 and $+1$): it achieves the value -1 at infinitely many points. The bracketing refinement methods (Fibonacci, golden section, quadratic fit) in this chapter apply only to **unimodal** functions. For multimodal functions, different strategies (such as the Shubert-Piyavskii method, also in this chapter) are needed.

Unimodality gives us a powerful *elimination rule*: if we find three points a, b, c arranged in increasing order (so $a < b < c$, meaning a is to the left of b, which is to the left of c) such that the *middle* value is the lowest, then the minimum must lie strictly between the outer two points.

3.1 Unimodality: The Key Assumption IV

Definition: Bracketing Triple

Given a unimodal function f, an ordered triple of points (a, b, c) with $a < b < c$ is called a **bracketing triple** if:

$$f(a) > f(b) \qquad \text{and} \qquad f(b) < f(c).$$

The two conditions together say that the middle point b produces a *lower* function value than both outer points a and c. The notation $f(a)$ means "the function f evaluated at the point a". When both inequalities hold, the global minimum of f is guaranteed to lie inside the open interval (a, c) — that is, somewhere strictly between a and c.

Notation for Bracketing Triple

a (left endpoint): the leftmost of the three points, a real number.
b (middle point): the interior point that achieves the lowest function value; $a < b < c$.
c (right endpoint): the rightmost point; $c > b > a$.
$f(a), f(b), f(c)$: the function values at the three points. These are real numbers (positive, negative, or zero).
(a, c) with round brackets (as opposed to $[a, c]$ with square brackets): the *open* interval, meaning all x with $a < x < c$ (endpoints excluded).

Why does the bracketing triple guarantee the minimum is in (a, c)? Because f is unimodal, the minimum x^* must be to the right of a (since $f(a) > f(b)$ means we are still descending as we move right from a) and to the left of c (since $f(c) > f(b)$ means we are still descending as we move left from c). Both conditions together trap the minimum in (a, c).

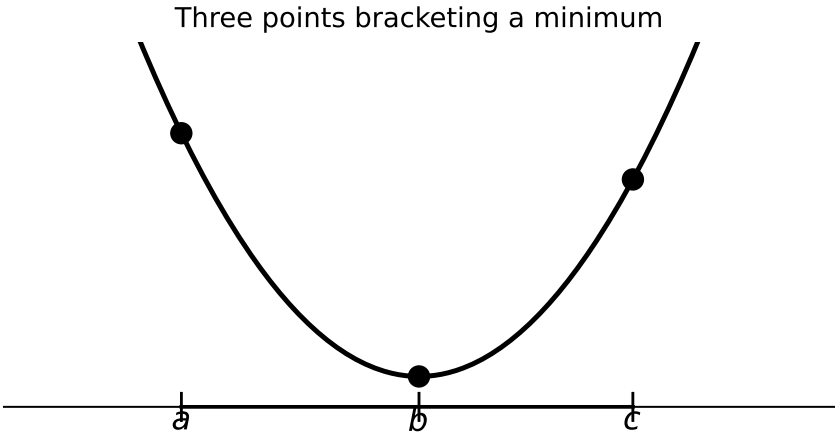
Key Insight: Why Unimodality Makes Bracketing Safe

The unimodality assumption is what makes every evaluation informative: once we have a valid triple (a, b, c) with $f(a) > f(b) < f(c)$, every subsequent evaluation either tightens the bracket from the left or from the right, but never risks losing the minimum. Without unimodality, the minimum could escape through a "back door" — a second valley hidden outside our current bracket.

Visualizing a Bracketing Triple I

The figure below shows a unimodal function with a valid bracketing triple (a, b, c) marked on the horizontal axis.

Visualizing a Bracketing Triple II



Visualizing a Bracketing Triple III

Reading the figure: the horizontal axis (the x-axis) shows the input values x. The vertical axis (the y-axis) shows the output values $f(x)$. The three points a, b, c are marked on the x-axis with dots directly above them showing the corresponding function values $f(a)$, $f(b)$, $f(c)$. Because the dot above b is the lowest of the three — that is, $f(b) < f(a)$ and $f(b) < f(c)$ — the triple is a valid bracket. The shaded region between a and c is the interval within which we continue to search. The unshaded regions to the left of a and to the right of c have been eliminated: the minimum cannot lie there (by the unimodality argument above).

3.2 Finding an Initial Bracket I

Before we can apply any of the refinement methods (Fibonacci, golden section, quadratic fit), we need a valid starting bracketing triple (a, b, c). This section describes a simple but effective algorithm that *finds* an initial bracket by probing the function, without any prior knowledge of where the minimum is.
The strategy is intuitive: start at some initial point x_0 (read "x sub zero"; the subscript 0 indicates the starting, or zeroth, iteration) and take a small step in the positive direction. If the function value goes *up* after that step, the minimum is to the left of x_0, so we reverse direction. We then march forward, doubling the step size at each iteration, until we find a point where the function starts going back up. At that moment we have a bracket: the current point is higher than the previous point, which is higher than the one before that, forming a valid triple.
The step doubling is the crucial design choice: it ensures the algorithm reaches the minimum in a logarithmic number of steps even if the minimum is far away, rather than crawling at constant speed.

3.2 Finding an Initial Bracket II

Algorithm 3.1 — `bracket_minimum`$(f,\ x_0;\ s,\ k)$

Inputs:

- f — the objective function to minimise (accepts a real number, returns a real number)
- x_0 (x sub zero) — the starting point; default value 0
- s — the initial step size (a small positive real number); default 10^{-2} (one hundredth)
- k (kappa, pronounced "kap-ah") — the step expansion factor, a multiplier greater than 1; default value 2

Output: an interval $[a, b]$ that brackets a local minimum.

Procedure:

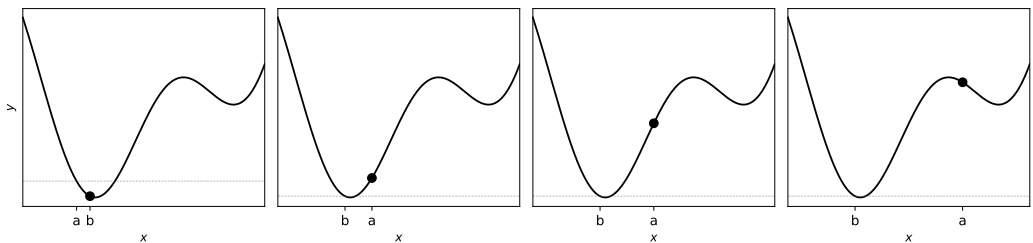
1. Set $a \leftarrow x_0$ and compute $y_a \leftarrow f(a)$. Then set $b \leftarrow a + s$ and compute $y_b \leftarrow f(b)$. (The arrow $\leftarrow$ means "assign the right-hand value to the left-hand variable".)
2. If $y_b > y_a$ (the step went uphill): swap a and b (so now a is the right point and b is the left), swap y_a and y_b, and negate s (set $s \leftarrow -s$) so we now march leftward.
3. **Repeat** until a bracket is found:
   - Compute $c \leftarrow b + s$ and $y_c \leftarrow f(c)$.
   - If $y_c > y_b$ (the function went back uphill): the minimum is between the last three points. **Return** the

Python: bracket_minimum

```python
import numpy as np

def bracket_minimum(f, x=0.0, s=1e-2, k=2.0):
    """Find an interval [a, b] that brackets a local minimum of f.

    Parameters
    ----------
    f : callable  -- univariate objective function
    x : float     -- starting point (default 0)
    s : float     -- initial step size (default 1e-2)
    k : float     -- step expansion factor, k > 1 (default 2)

    Returns
    -------
    (a, b) : tuple of floats with a < b
    """
    a, ya = x, f(x)
    b, yb = a + s, f(a + s)

    if yb > ya:                       # first step went uphill -- reverse
        a, b = b, a
        ya, yb = yb, ya
        s = -s

    while True:
        c, yc = b + s, f(b + s)
        if yc > yb:                   # found the bracket -- return it
```

Bracket Minimum: Seeing the Algorithm in Action I

The figure below shows four successive stages of the `bracket_minimum` algorithm applied to a sample function. Each panel corresponds to one decision made by the algorithm.



Bracket Minimum: Seeing the Algorithm in Action II

Reading the panels from left to right:
Panel 1 (leftmost): The algorithm takes an initial step to the right from the starting point x_0. It finds that the function value went *up* (the step was uphill), so it reverses direction by setting $s \leftarrow -s$ and begins marching to the left.
Panels 2 and 3: Each subsequent step covers $k = 2$ times the distance of the previous one. The function values are still decreasing as we march left, so no bracket has been found yet. Notice the growing arrow lengths: the algorithm is accelerating geometrically.
Panel 4 (rightmost): The algorithm takes a step and the function value *increases* for the first time. This signals that the minimum lies somewhere between the second-to-last and last positions. Together with the position before that, a valid bracketing triple is formed and the algorithm returns the bounding interval.
The key observation: the step size doubling (with $k = 2$) means that even if the minimum is very far from x_0, the algorithm needs only a logarithmic number of steps. Specifically, if the minimum is at distance D from x_0 and the initial step size is s, the algorithm terminates in approximately $\log_2(D/s)$ (log base 2 of D over s) iterations.

3.3 Fibonacci Search — The Core Idea I

Once we have a valid bracketing interval $[a, b]$, the next question is: how do we shrink it as efficiently as possible? **Fibonacci search** gives the provably optimal answer for a *fixed budget* of n function evaluations on a unimodal function.

The central observation is that at each step, we place two interior query points inside the current interval. After comparing the two function values, we discard one side of the interval. Crucially, *one* of the two points is reused in the next step — we already know its function value — so we only need *one new evaluation* per step after the first. The question is: where exactly should we place the two points to maximise the guaranteed interval shrinkage after all n evaluations?

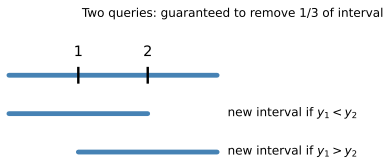
The Two-Query Optimal Placement

Suppose we have a unimodal f on $[a, b]$ and are allowed exactly *two* evaluations. Placing them at the $\frac{1}{3}$ and $\frac{2}{3}$ positions — that is, at $a + \frac{1}{3}(b - a)$ and $a + \frac{2}{3}(b - a)$ — guarantees that regardless of which value is lower, we can discard exactly $\frac{1}{3}$ of the interval. No two-query strategy can guarantee discarding more than $\frac{1}{3}$. However, if we are allowed to place the two points infinitesimally close together at the center, we can discard nearly *half* the interval — but this only works when we have effectively infinite evaluations remaining to resolve which side the minimum is on.

3.3 Fibonacci Search — The Core Idea II

Fibonacci search generalises this trade-off: the optimal placement ratio for n total evaluations is determined by the ratio of two consecutive Fibonacci numbers.

As the total budget n increases, the placement ratio for each step approaches $\varphi^{-1} \approx 0.618$ (phi to the minus one, read "one over phi", where φ (phi, pronounced "fee" or "fye") is the golden ratio). This observation leads directly to golden section search, discussed in the next section.



The key saving: one of the two interior evaluation points from the current step becomes an interior point of the *next* step. This means we only pay for one *new* evaluation per step, not two.

3.3 Fibonacci Search — The Core Idea III

Key Insight: Fibonacci Search Is Provably Optimal

Given a fixed budget of n function evaluations on a unimodal function, no deterministic algorithm can guarantee a smaller final interval than Fibonacci search. The final interval has length I_1/F_{n+1}, where $I_1 = b - a$ is the initial interval length and F_{n+1} (F sub n plus one) is the $(n+1)$-th Fibonacci number. Because Fibonacci numbers grow exponentially, more evaluations always help — but with diminishing returns.

Fibonacci Numbers and Why They Control Shrinkage I

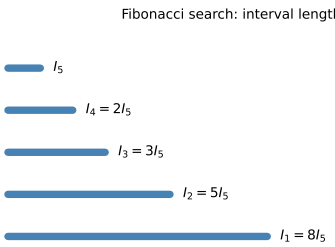
To understand *why* Fibonacci numbers determine the optimal shrinkage, we first review the sequence itself and its key properties.

The Fibonacci Sequence (Recurrence)

The Fibonacci sequence is defined by the following rule (a *recurrence*, meaning each term is defined in terms of previous terms):

$$F_n = \begin{cases} 1 & \text{if } n \leq 2 \\ F_{n-1} + F_{n-2} & \text{if } n > 2 \end{cases}$$

Reading the formula: F_n (F sub n) is the n-th term in the sequence, where n is a positive integer index starting at 1. Each term is the sum of the two immediately preceding terms. The first several values are: $F_1 = 1$, $F_2 = 1$, $F_3 = 2$, $F_4 = 3$, $F_5 = 5$, $F_6 = 8$, $F_7 = 13$, $F_8 = 21$, $F_9 = 34$, $F_{10} = 55, \ldots$



Each row shows the remaining bracket after one more evaluation. The key relation is $I_1 = F_{n+1} \cdot I_n$, where I_k is the interval length at step k (counting from the end), and I_n is the final interval length.

Fibonacci Numbers and Why They Control Shrinkage II

Binet's Closed-Form Formula

The Fibonacci numbers can be computed directly — without applying the recurrence n times — using **Binet's formula**:

$$F_n = \frac{\varphi^n - (1-\varphi)^n}{\sqrt{5}}, \qquad \varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$$

Here φ (phi, the golden ratio) and $\sqrt{5}$ (the square root of five, approximately 2.236) are the two key constants.

Fibonacci Numbers and Why They Control Shrinkage III

Notation for Binet's Formula

F_n (F sub n): the n-th Fibonacci number, a positive integer.
φ (phi, pronounced "fee" or "fye"): the golden ratio, approximately 1.618. It satisfies the remarkable algebraic property $\varphi^2 = \varphi + 1$, which is the source of many of its special properties in geometry, art, and mathematics.
φ^n (phi to the power n): the golden ratio raised to the n-th power. This term grows without bound as n increases.
$(1-\varphi)^n$: the second term, approximately $(-0.618)^n$. Since $|1-\varphi| \approx 0.618 < 1$, repeated multiplication drives this term toward zero, so for large n, $F_n \approx \varphi^n/\sqrt{5}$.
$\sqrt{5}$ (square root of five): the normalising constant.

Fibonacci Numbers and Why They Control Shrinkage IV

How to Read Binet's Formula

The formula tells us that Fibonacci numbers grow *exponentially*, at the rate of $\varphi \approx 1.618$ per step. This means that every additional evaluation of Fibonacci search shrinks the interval by a factor of approximately 1.618 — equivalent to about 70% of the previous length. Ten additional evaluations shrink the interval by $\varphi^{10} \approx 123$ times.

Ratio of Successive Fibonacci Numbers

The exact fractional placement used at step n of Fibonacci search is:

$$\rho_n = \frac{F_n}{F_{n+1}} = \varphi^{-1}\,\frac{1 - s^{n+1}}{1 - s^n}, \qquad s = \frac{1-\sqrt{5}}{1+\sqrt{5}} \approx -0.382$$

where ρ_n (rho sub n, pronounced "row") is the fraction of the interval from the left endpoint at which the interior point is placed, and s (a small constant, approximately -0.382) is a correction factor that accounts for the difference between the exact Fibonacci ratio and the limiting golden ratio. As $n \to \infty$ (as n grows without bound), $\rho_n \to \varphi^{-1} \approx 0.618$, confirming that golden section search uses the limiting ratio.

Algorithm 3.2: Fibonacci Search in Python

```python
import numpy as np

def fibonacci_search(f, a, b, n, eps=0.01):
    """Fibonacci search: find minimum of unimodal f on [a, b].

    n   : total number of function evaluations (n > 1)
    eps : small tolerance for the final disambiguation step
    Returns the bracketing interval (a, b) after n evaluations.
    """
    phi = (1 + np.sqrt(5)) / 2            # golden ratio, ~1.618
    s   = (1 - np.sqrt(5)) / (1 + np.sqrt(5))  # correction, ~-0.382

    # Placement ratio for the first (right) interior point
    rho = 1.0 / (phi * (1 - s**(n+1)) / (1 - s**n))
    r   = (1 - rho) * a + rho * b         # right interior point
    yr, n_left = f(r), n - 1              # evaluate f; one eval used

    while n_left > 0:
        if n_left > 1:
            lam = rho * a + (1 - rho) * b     # left interior point
        else:
            lam = eps * a + (1 - eps) * r     # last step: near center

        rho     = 1.0 / (phi * (1 - s**(n_left+1)) / (1 - s**n_left))
        yl      = f(lam)                       # one new evaluation
        n_left -= 1

        if yl < yr:                            # left is lower
```

Fibonacci Search Code Walkthrough I

Let us trace through the key parts of the Python implementation to build intuition for what each line accomplishes.

Lines 1–2 (Setup constants): We compute φ (phi) as $(1+\sqrt{5})/2 \approx 1.618$ and the correction factor $s \approx -0.382$. These two constants, combined with the number of remaining evaluations, give the exact Fibonacci-optimal placement ratio ρ (rho, pronounced "row") at each step.

Placement ratio ρ (rho): The variable `rho` holds the fractional distance from the left endpoint a to the right interior query point. For example, $\rho = 0.6$ means the point is placed 60% of the way from a to b. This ratio is re-computed at each step based on how many evaluations remain.

Interior points r and λ (lambda, pronounced "lam-da"): At each iteration the algorithm maintains two interior points: `r` (the "right" interior point) and `lam` (short for λ, the "left" interior point). The point `r` was evaluated in the previous step and its value `yr` is already known. Only `lam` requires a new call to f — this is the one-new-evaluation-per-step property.

Bracket update rule: After comparing $f(\lambda)$ (stored as `yl`) and $f(r)$ (stored as `yr`):

- If $f(\lambda) < f(r)$ (the left interior point is lower): the minimum must be in $[a, r]$, so we discard everything to the right of r by updating $b \leftarrow r$.
- Otherwise: the minimum is in $[\lambda, b]$, so we discard everything to the left of λ by updating $a \leftarrow \lambda$.

Fibonacci Search Code Walkthrough II

Last step (ε (epsilon, pronounced "ep-si-lon"), a very small positive number): The final iteration places a point very close to r (at fraction ε from a to r) to resolve which side of the center contains the minimum. The default $\texttt{eps} = 0.01$ is small enough for most purposes.

Example 3.1: Fibonacci Search on $f(x) = e^{x-2} - x$ I

We now work through a concrete numerical example step by step so you can see exactly how the interval shrinks with each evaluation.

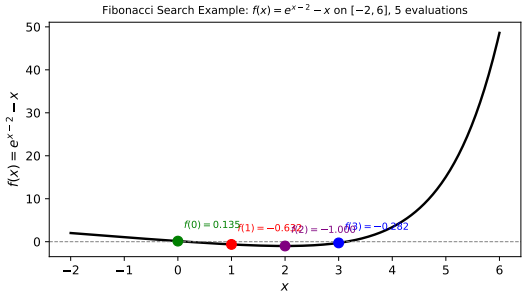
Problem Setup

We minimise $f(x) = e^{x-2} - x$ (read "e to the power x minus 2, minus x", where $e \approx 2.718$ is Euler's number) on the interval $[a, b] = [-2, 6]$ using exactly $n = 5$ function evaluations. The true minimum is at $x^* = 2$. (This can be verified by computing the derivative $f'(x) = e^{x-2} - 1$ and solving $f'(x) = 0$: $e^{x-2} = 1$ implies $x - 2 = 0$ so $x = 2$.)

The initial interval has length $I_1 = 6 - (-2) = 8$. With $n = 5$ evaluations, Fibonacci search guarantees the final interval has length at most $I_1 / F_{n+1} = 8/F_6 = 8/8 = 1$.

Example 3.1: Fibonacci Search on $f(x) = e^{x-2} - x$ II

Evaluation k	Query $x^{(k)}$	$f(x^{(k)})$ (approx)
$k = 1$	$x^{(1)} = 1.0$	$e^{-1} - 1 \approx -0.632$
$k = 2$	$x^{(2)} = 3.0$	$e^{1} - 3 \approx -0.282$
$f(1) < f(3)$; discard $[3, 6]$. New: $[-2, 3]$		
$k = 3$	$x^{(3)} = 0.0$	$e^{-2} - 0 \approx 0.135$
$f(0) > f(1)$; discard $[-2, 0]$. New: $[0, 3]$		
$k = 4$	$x^{(4)} = 2.0$	$e^{0} - 2 = -1.0$
$f(2) < f(1)$; discard $[0, 1]$. New: $[1, 3]$		
$k = 5$	near $2 + \varepsilon$	disambiguates center



Coloured dots show the evaluation order. The true minimum at $x^* = 2$ is reached exactly at evaluation 4 in this example.

Example 3.1: Fibonacci Search on $f(x) = e^{x-2} - x$ III

Notation in the Table

$x^{(k)}$ (x superscript k in parentheses): the x-value queried at the k-th evaluation. The parenthesised superscript distinguishes iteration counts from exponents (so $x^{(2)}$ means "second query point", not "x squared").
$f(x^{(k)})$: the function value returned by the objective at the k-th query.
$e^{-1} \approx 0.368$, so $e^{-1} - 1 \approx -0.632$.
$e^{1} \approx 2.718$, so $e^{1} - 3 \approx -0.282$.

Interpretation

After 5 evaluations we have narrowed the bracket to $[1, 3]$, an interval of length 2. The guarantee was length ≤ 1, so we actually did better here because the minimum happened to be found exactly. This illustrates a key point: the Fibonacci guarantee is a *worst-case* bound — in practice the method often converges faster.

3.4 Golden Section Search I

Fibonacci search is provably optimal but requires one practical decision up front: we must commit to the total number of evaluations n *before* the first evaluation. If we later decide we need more precision, we cannot simply continue the same sequence.

Golden section search removes this limitation. It uses a *constant* placement ratio at every step — the ratio $\varphi^{-1} \approx 0.618$ (one over phi) — which is the *limit* of the Fibonacci placement ratio as $n \to \infty$. Because the ratio never changes, we can add evaluations one at a time and stop whenever the bracket is small enough.

Key Formula: The Limiting Ratio (derivation of golden section)

As the total budget n grows without bound (written $n \to \infty$, read "n goes to infinity"), the Fibonacci placement ratio converges to the reciprocal of the golden ratio:

$$\lim_{n\to\infty} \frac{F_n}{F_{n+1}} = \frac{1}{\varphi} = \varphi - 1 = \frac{\sqrt{5}-1}{2} \approx 0.618$$

Golden section search uses this constant ratio $\rho = \varphi^{-1} \approx 0.618$ at every iteration, regardless of how many evaluations remain.

3.4 Golden Section Search III

Notation

$\lim_{n\to\infty}$ (read "the limit as n goes to infinity"): the value the expression approaches as n increases without bound.
F_n/F_{n+1} (F sub n divided by F sub n plus one): the ratio of two consecutive Fibonacci numbers; this ratio converges to $1/\varphi$.
φ (phi, the golden ratio): ≈ 1.618, satisfying $\varphi^2 = \varphi + 1$.
φ^{-1} (phi to the minus one, read "one over phi"): the reciprocal of the golden ratio, ≈ 0.618. Remarkably, $\varphi^{-1} = \varphi - 1$.
$\sqrt{5}$ (square root of five): ≈ 2.236.

Using the constant ratio $\rho = \varphi^{-1} \approx 0.618$ at every step:

- The interval is always split in the same golden proportion, so the algorithm has a regular, predictable structure.
- After n evaluations, the bracket length has shrunk by the factor φ^{n-1}: an initial interval of length I_1 becomes an interval of length $I_1 \cdot \varphi^{-(n-1)}$.

3.4 Golden Section Search IV

- To guarantee the minimum is located within a desired tolerance ε (epsilon, the target precision — the minimum acceptable interval length):

$$n = \left\lceil \frac{\ln\big((b-a)/\varepsilon\big)}{\ln \varphi} \right\rceil \quad \text{evaluations are sufficient.}$$

3.4 Golden Section Search V

Notation for the Evaluation-Count Formula

$\ln(\cdot)$ (the natural logarithm, read "ell-en" or "log"): the inverse of the exponential function $e^{(\cdot)}$. It converts the exponential shrinkage factor into a linear count of iterations.
$(b - a)$: the initial interval length.
ε (epsilon): the desired final interval length (precision).
$(b - a)/\varepsilon$: the ratio of initial to final interval length — how much total shrinkage is needed.
$\lceil \cdot \rceil$ (the "ceiling" function, read "ceiling of"): round up to the nearest integer, since we need a whole number of evaluations.
$\ln \varphi \approx 0.481$: the natural log of the golden ratio, which appears because each evaluation shrinks the interval by φ.

3.4 Golden Section Search VI

Worked Example: How Many Evaluations Are Needed?

Suppose $[a, b] = [0, 100]$ (initial interval length $= 100$) and we want precision $\varepsilon = 0.001$ (one thousandth). Substituting:

$$n = \left\lceil \frac{\ln(100/0.001)}{\ln(1.618)} \right\rceil = \left\lceil \frac{\ln(100{,}000)}{0.481} \right\rceil = \left\lceil \frac{11.51}{0.481} \right\rceil = \lceil 23.93 \rceil = 24.$$

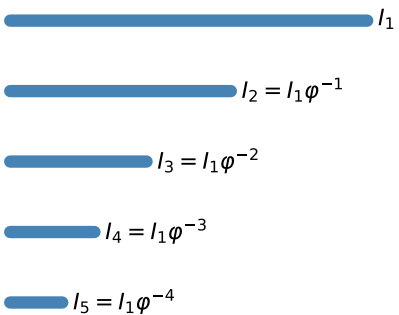
So 24 evaluations are sufficient to narrow a search region of length 100 down to 0.001 — a reduction by a factor of 100,000. This illustrates the remarkable efficiency of golden section search.

Practical Advantage: The Anytime Property

Because the placement ratio is always the same constant φ^{-1}, we can **stop at any time** and the current bracket is always valid. We do not need to commit to a budget in advance. After each evaluation we check whether the bracket is small enough; if yes, we stop; if no, we continue. This "anytime" property makes golden section search the method of choice when the required precision is not known in advance.

Golden Section: How the Interval Shrinks I

Golden section: shrinks by φ^{n-1} after n queries



Golden Section: How the Interval Shrinks II

This figure shows the bracketing interval shrinking with each successive evaluation. Each horizontal bar represents the current bracket; the two interior marks on each bar show the positions of the two interior query points. After each evaluation, the portion of the bar to the far side of the *higher* interior point is discarded, and the bar shortens.
Notice the critical reuse property: at every step, *one* of the two interior points from the current bar becomes an interior point in the next bar (its function value is already known from the previous iteration). Only one new call to f is needed per step. The left panel confirms this: the surviving interior point from step k always sits at the golden ratio position inside the new, shorter bar at step $k+1$.

Key Insight: Asymptotic Equivalence with Fibonacci

For large n, the interval shrinkage per step is $\varphi^{-1} \approx 0.618$ for both methods. The difference is that for small n (say $n \leq 6$) Fibonacci uses the exact optimal ratios (slightly non-constant) while golden section uses the limiting constant ratio — so Fibonacci is a tiny bit better for very small budgets. In practice this difference is negligible.

Algorithm 3.3: Golden Section Search in Python

```python
import numpy as np

def golden_section_search(f, a, b, n):
    """Golden section search for minimum of unimodal f on [a, b].

    n  : total number of function evaluations (n > 1)
    Returns the bracketing interval (a, b) after n evaluations.
    """
    phi = (1 + np.sqrt(5)) / 2    # golden ratio, ~1.618
    rho = phi - 1                  # = 1/phi ~ 0.618 (placement ratio)

    # Place the first interior point (called d) at fraction rho from left
    d  = rho * b + (1 - rho) * a
    yd = f(d)                      # one evaluation used

    for _ in range(n - 1):
        # Place the symmetric interior point c
        c  = rho * a + (1 - rho) * b
        yc = f(c)                  # one new evaluation per step

        if yc < yd:                # left interior point is lower
            b, d, yd = d, c, yc # discard [d, b]; new bracket [a, d]
        else:                      # right interior point is lower (or equal)
            a, b = b, c           # discard [a, c]; new bracket [c, b]
            # Note: d and yd are retained as the new right interior point

    return (a, b) if a < b else (b, a)
```

Golden Section vs. Fibonacci: When to Use Each I

Both methods shrink the bracket at essentially the same rate. The choice between them comes down to practical considerations.

Fibonacci Search

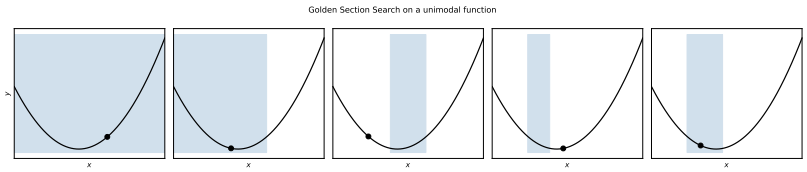
- **Provably optimal** for a fixed, predetermined budget of n evaluations: no other method can guarantee a smaller final interval.
- The placement ratio ρ_n changes at every step — it depends on how many evaluations remain.
- Requires knowing n before the first evaluation.
- After n evaluations, the interval has shrunk by the exact factor F_{n+1} (the $(n+1)$-th Fibonacci number).

Golden Section Search

- **Near-optimal** for large n; for small n the approximation error is tiny and practically irrelevant.
- The placement ratio is the constant $\rho = \varphi^{-1} \approx 0.618$ at every step — no bookkeeping required.
- Can be run as an *anytime* algorithm: stop whenever the bracket is small enough.
- After n evaluations, the interval has shrunk by the factor φ^{n-1} relative to the original length.

Golden Section vs. Fibonacci: When to Use Each II

The figure below shows the golden section evaluation sequence on a unimodal function. The shaded region marks the current bracket, and coloured dots show the evaluated points in order. Notice how the bracket tightens around the minimum with each new dot.



Practical Recommendation

In the vast majority of real applications, **use golden section search**. The difference in performance versus Fibonacci search is less than 1% for $n > 10$, and the constant-ratio simplicity means you can stop early, restart, or run adaptively without redesigning the algorithm. Reserve Fibonacci search for situations where you have a fixed and strict budget constraint and need the absolute best worst-case guarantee.

3.5 Quadratic Fit Search — Motivation I

Both golden section and Fibonacci search treat the function f as a complete black box. They make no assumption about the *shape* of f, and they converge at a geometric rate — the error decreases by a constant factor $\varphi^{-1} \approx 0.618$ per evaluation.
Can we do better by exploiting shape information? Yes — if we assume the function is **smooth** (meaning it has continuous derivatives near the minimum), then by Taylor's theorem, any smooth function looks like a *quadratic* (a parabola of the form $ax^2 + bx + c$) near its minimum. This is because the Taylor expansion around x^* gives:

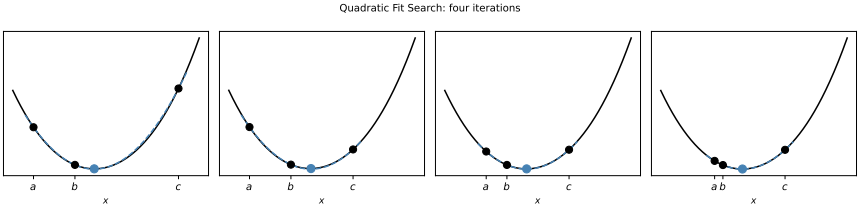
$$f(x) \approx f(x^*) + \tfrac{1}{2} f''(x^*)(x - x^*)^2 + \dots$$

(read "f of x approximately equals f of x-star plus one-half times f double-prime of x-star times the square of x minus x-star"). Here $f''(x^*)$ (f double-prime, the second derivative of f at x^*) is a positive constant for a minimum, so the dominant behaviour is indeed quadratic.

3.5 Quadratic Fit Search — Motivation II

Key Insight: Fit a Parabola, Jump to Its Vertex

Given three bracketing points, we can fit a unique parabola through all three (there is exactly one quadratic that passes through three given points). The vertex (lowest point) of this parabola can be computed *exactly* from a simple algebraic formula. Jumping to the vertex rather than taking a cautious golden-section step produces much faster convergence near the minimum.



3.5 Quadratic Fit Search — Motivation III

Reading the figure: Black dots mark the three current bracketing points. The blue curve is the fitted parabola passing through all three. The blue dot is the vertex of the parabola — the predicted next query point. Notice how closely the parabola tracks the true function (grey curve) near the minimum.

The convergence is typically **super-linear**: the number of correct decimal digits grows faster than linearly with each iteration. For a perfectly quadratic function, the method finds the minimum *exactly* in a single step.

One important caveat: safeguards are needed when the predicted point falls outside the bracket, or when the three points are nearly collinear (nearly on a straight line), which would make the fitted parabola nearly flat and the vertex prediction unstable.

Quadratic Fit: Deriving the Minimiser Formula I

Given three bracketing points $a < b < c$ with known function values $y_a = f(a)$, $y_b = f(b)$, $y_c = f(c)$, we derive the unique quadratic that passes through them and find its minimum analytically.

Step 1: The Unique Quadratic via Lagrange Interpolation

The unique quadratic $q(x)$ (q of x, our parabola model) passing through the three points (a, y_a), (b, y_b), (c, y_c) is:

$$q(x) = y_a \cdot \frac{(x-b)(x-c)}{(a-b)(a-c)} + y_b \cdot \frac{(x-a)(x-c)}{(b-a)(b-c)} + y_c \cdot \frac{(x-a)(x-b)}{(c-a)(c-b)}$$

Each of the three terms is a *Lagrange basis polynomial*: a quadratic that equals 1 at one of the three nodes and 0 at the other two. Multiplied by the corresponding function value and summed, they interpolate f exactly at all three points.

Quadratic Fit: Deriving the Minimiser Formula II

Notation for the Lagrange Formula

$q(x)$: the fitted quadratic function, our model of f near the minimum.
a, b, c: the three query x-coordinates, with $a < b < c$.
y_a, y_b, y_c: the function values $f(a)$, $f(b)$, $f(c)$ at those points.
$(x - b)(x - c)$: a product of two linear expressions. This equals zero when $x = b$ and when $x = c$, ensuring those points are handled by the other two terms.
$(a - b)(a - c)$: the denominator for the first basis polynomial, a normalising constant ensuring the term equals 1 at $x = a$.

Quadratic Fit: Deriving the Minimiser Formula III

Step 2: Finding the Minimiser — Setting $q'(x^*) = 0$

The derivative $q'(x)$ of a quadratic is a linear function, so setting it to zero gives a unique solution. After algebra, the minimiser of $q(x)$ is:

$$x^* = \frac{1}{2} \cdot \frac{y_a(b^2 - c^2) + y_b(c^2 - a^2) + y_c(a^2 - b^2)}{y_a(b - c) + y_b(c - a) + y_c(a - b)}$$

This gives the x-coordinate of the parabola's vertex — the point where the fitted quadratic $q(x)$ is smallest.

Quadratic Fit: Deriving the Minimiser Formula IV

Notation for the Minimiser Formula

x^* (x star): the predicted minimiser, the next query point.
The **numerator** $y_a(b^2 - c^2) + y_b(c^2 - a^2) + y_c(a^2 - b^2)$: a weighted combination of the squared positions of the three points, where the weights are the function values. It has the same units as $y_a \cdot x^2$.
The **denominator** $y_a(b - c) + y_b(c - a) + y_c(a - b)$: a weighted combination of the gaps between points, with units of $y_a \cdot x$. If the denominator is zero, the three points are collinear (lie on a straight line) and no unique quadratic minimum exists. In practice, this is handled by a guard check.
The factor $\frac{1}{2}$ scales the numerator-over-denominator ratio to give the correct vertex position.

Quadratic Fit: Deriving the Minimiser Formula V

How to Read the Minimiser Formula

Think of x^* as a weighted average of the three points a, b, c, where the weights depend on the function values. If y_b is much smaller than y_a and y_c (the middle point is much lower than the ends), the formula pulls x^* close to b. If the true f is exactly a parabola, this formula gives the *exact* answer in one step. For functions that are only approximately quadratic near the minimum, several iterations bring the bracket to high precision — but each iteration reduces the error faster than any fixed geometric ratio, which is what we call "super-linear convergence".

Algorithm 3.4: Quadratic Fit Search in Python

```python
import numpy as np

def quadratic_fit_search(f, a, b, c, n):
    """Quadratic fit search for minimum of smooth f.

    a < b < c  : initial bracketing triple (a, b, c) with f(b) < f(a), f(c)
    n          : total evaluations including the initial 3 (so n >= 3)
    Returns (a, b, c) as the final bracketing triple.
    """
    ya, yb, yc = f(a), f(b), f(c)

    for _ in range(n - 3):          # n-3 additional evaluations
        # Compute the vertex of the fitted quadratic
        denom = ya * (b - c) + yb * (c - a) + yc * (a - b)
        x = 0.5 * (ya * (b**2 - c**2) +
                   yb * (c**2 - a**2) +
                   yc * (a**2 - b**2)) / denom
        yx = f(x)                   # evaluate at the predicted minimiser

        if x > b:                   # x lies between b and c
            if yx > yb:             # f went up from b to x
                c, yc = x, yx       # tighten from right
            else:                   # f went down from b to x
                a, ya, b, yb = b, yb, x, yx
        elif x < b:                 # x lies between a and b
            if yx > yb:             # f went up from b back to x
                a, ya = x, yx       # tighten from left
            else:                   # f went down from b to x
```

Quadratic Fit: A Worked Numerical Example I

We now trace through the formula with explicit numbers to build confidence that the algebra is correct and to see how the method converges.

Problem Setup

Minimise $f(x) = (x-1)^2 + 1$ (read "the quantity x minus 1, squared, plus 1") starting with the bracketing triple $(a, b, c) = (0,\ 1,\ 3)$. The true minimum is at $x^* = 1$ with $f(1) = 0 + 1 = 1$.

Step 1: Compute function values.

$$y_a = f(0) = (0-1)^2 + 1 = (-1)^2 + 1 = 1 + 1 = 2$$
$$y_b = f(1) = (1-1)^2 + 1 = 0 + 1 = 1$$
$$y_c = f(3) = (3-1)^2 + 1 = 4 + 1 = 5$$

Step 2: Compute the denominator D.

$$D = y_a(b-c)+y_b(c-a)+y_c(a-b) = 2(1-3)+1(3-0)+5(0-1) = 2(-2)+1(3)+5(-1) = -4+3-5 = -6$$

Quadratic Fit: A Worked Numerical Example II

Step 3: Compute the numerator N.

$$N = y_a(b^2-c^2)+y_b(c^2-a^2)+y_c(a^2-b^2) = 2(1-9)+1(9-0)+5(0-1) = 2(-8)+9+5(-1) = -16+9-5 = -$$

Step 4: Apply the formula.

$$x^* = \frac{1}{2} \cdot \frac{N}{D} = \frac{1}{2} \cdot \frac{-12}{-6} = \frac{1}{2} \cdot 2 = 1.0$$

Result and Interpretation

The quadratic fit formula returns $x^* = 1.0$, which is the *exact* minimiser, found in a single step. This is not a coincidence: the true function $f(x) = (x-1)^2 + 1$ is itself a perfect quadratic, so the fitted parabola *is* the true function, and its vertex is found exactly. In practice, for functions that are only approximately quadratic near the minimum (smooth but not exactly parabolic), several iterations are needed — but each one converges super-linearly.

Quadratic Fit: A Worked Numerical Example III

When the Method Can Struggle

If the denominator D is very close to zero — indicating the three points are nearly collinear — the formula becomes numerically unstable (dividing by a very small number amplifies rounding errors). Robust implementations add a guard: if $|D| < \delta$ (delta, a small threshold), fall back to a golden section step instead.

3.6 Shubert-Piyavskii Method — Motivation I

All the methods so far — Fibonacci, golden section, quadratic fit — assume the function is **unimodal**: exactly one valley. In many real-world problems the objective has multiple local minima (it is *multimodal*), and we need the *global* minimum, not just a local one that happens to be nearby.
The **Shubert-Piyavskii method** (named after its inventors, Boris Shubert and Shubert Piyavskii) addresses this harder problem. Its key idea is to maintain a **global lower bound** on f across the entire interval $[a, b]$, and always sample the point where this lower bound is *most optimistic* (lowest) — since that is where the global minimum might be hiding.
The mathematical tool that makes this possible is **Lipschitz continuity**, which bounds how fast the function can change from one point to another.

Definition: Lipschitz Continuity

A function f is said to be **Lipschitz continuous** on the interval $[a, b]$ with **Lipschitz constant** ℓ (lowercase letter L, pronounced like the letter "ell") if the following inequality holds for every pair of points x and y in $[a, b]$:

$$|f(x) - f(y)| \leq \ell\,|x - y| \qquad \forall\, x,\, y \in [a, b].$$

Notation for Lipschitz Continuity

$|\cdot|$ (absolute value, the vertical bars): the distance from zero, making the quantity always non-negative. So $|f(x) - f(y)|$ is the (non-negative) difference in function values, and $|x - y|$ is the (non-negative) distance between the two input points.

ℓ (ell): the Lipschitz constant. It is an upper bound on the magnitude of the slope $|f'(x)|$ at every point in $[a, b]$. Intuitively, ℓ is the maximum speed (in function-value units per x-unit) at which f can rise or fall.

$\forall$ (upside-down A, read "for all"): the inequality must hold for every possible pair (x, y), not just one particular pair.

$x, y \in [a, b]$ (read "x and y are in the interval $[a, b]$"): both points lie within the interval we are optimising over.

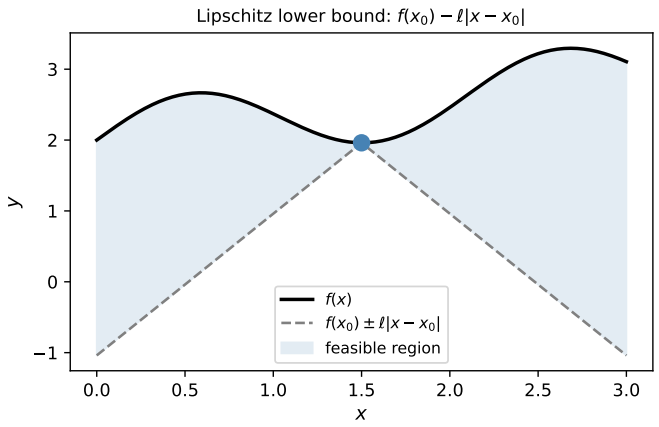
How to Read the Lipschitz Condition: The Ruler Analogy

Imagine drawing the function on paper. Place a ruler angled at slope $+\ell$ or $-\ell$ at any sampled point. The Lipschitz condition guarantees the true function *never rises or falls faster than the ruler line*. From any sampled point $(x_0, f(x_0))$, the function must lie *above* the "V-shaped tent" formed by two lines of slope $+\ell$ and $-\ell$ radiating outward from that point:

$$f(x) \geq f(x_0) - \ell\,|x - x_0| \quad \text{for all } x \in [a, b].$$

This V-shaped lower bound is the building block of the Shubert-Piyavskii method. Combining the V-shapes from all sampled points gives a *sawtooth* lower bound on the entire function.

The Lipschitz Lower Bound: From a Single Point I



Reading the figure: At the sampled point x_0 we draw two straight lines emanating outward:

- A line of slope $+\ell$ going to the right: the function can increase at most this fast as we move right from x_0.
- A line of slope $-\ell$ going to the left: the function can increase at most this fast as we move left from x_0.

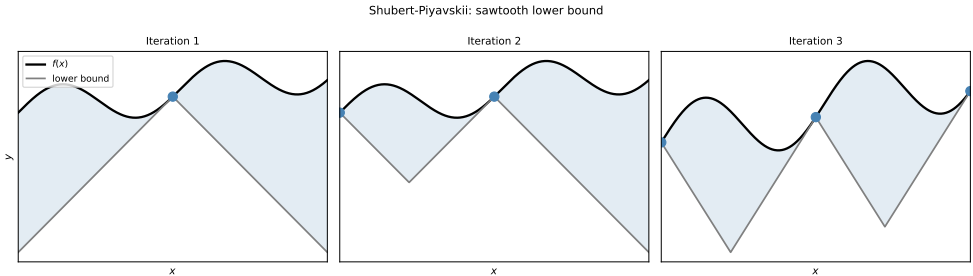
The region *below* both lines (the shaded V beneath the two dashed lines) is where the function is guaranteed *never to be*. The function must lie *above* the V at every point.

The bottom of the V at any point x provides a lower bound on $f(x)$. As we accumulate more sampled points, each gives its own V-shape. The *tightest* lower bound at any given x is the *maximum* of all the

The Lipschitz Lower Bound: From a Single Point II

individual V-shapes from all sampled points — since the maximum of a set of lower bounds is itself a lower bound, and a tighter one. This maximum-of-V-shapes is the characteristic *sawtooth* pattern of the Shubert-Piyavskii method.

Shubert-Piyavskii: The Sawtooth Lower Bound I



This figure shows the sawtooth lower bound building up across successive iterations of the algorithm. Reading the panels:

After Iteration 1: The algorithm samples the midpoint of $[a, b]$. A single V-shape radiates from this point, giving a crude lower bound that covers the entire interval.

Shubert-Piyavskii: The Sawtooth Lower Bound II

After Iteration 2: The next sample is placed at the *global minimum of the current sawtooth* — the lowest point of the V-shape, where the lower bound is most optimistic. Two V-shapes now combine to form a two-tooth sawtooth; the bound is sharper.
After Iteration 3: The sawtooth sharpens further. More peaks approach the true function, and more valleys are elevated upward.
Termination: The algorithm stops when the *optimality gap* δ (delta, pronounced "del-ta", meaning the gap between the best function value found and the current sawtooth minimum) satisfies:

$$\delta = f_{\text{best}} - y_{\text{saw,min}} \leq \varepsilon$$

where f_{best} is the smallest f value seen so far, $y_{\text{saw,min}}$ is the minimum of the sawtooth lower bound, and ε (epsilon) is the desired tolerance. When this gap is small, we know the global minimum cannot be much better than f_{best}.

Uncertainty Regions After Each Evaluation I

After each set of evaluations, the Shubert-Piyavskii method can quantify exactly which sub-regions of $[a, b]$ could still contain the global minimum.

Uncertainty Region Formula

For the i-th sawtooth vertex at position $x^{(i)}$ (x superscript i, the i-th lower vertex of the sawtooth) with sawtooth value $y^{(i)}$ (the lower bound at that vertex), and for the best function value found so far $y_{\min}$ (the smallest f evaluated at any sampled point), the **uncertainty region** around the i-th vertex is:

$$\left[x^{(i)} - \frac{y_{\min} - y^{(i)}}{\ell},\ x^{(i)} + \frac{y_{\min} - y^{(i)}}{\ell}\right]$$

This region is non-empty only when $y^{(i)} < y_{\min}$, meaning the sawtooth lower bound dips below the best sampled value — so there is still hope for improvement in that region.

Uncertainty Regions After Each Evaluation II

Notation for the Uncertainty Region

$x^{(i)}$ (x superscript i, in parentheses to distinguish from exponentiation): the horizontal position of the i-th sawtooth vertex.
$y^{(i)}$ (y superscript i): the sawtooth lower bound value at $x^{(i)}$.
$y_{\min}$: the smallest function value evaluated at any sampled point so far. This is the "best solution we have found".
$y_{\min} - y^{(i)}$ (y-min minus y superscript i): how far the sawtooth dips below the best known value. A larger dip means the global minimum might be significantly better than what we have found, so we cannot rule out that region.
ℓ (ell): the Lipschitz constant. A larger ℓ (the function can change faster) means a narrower uncertainty region: the slope constraint restricts how far from $x^{(i)}$ the improvement could be.
Dividing by ℓ converts a "depth" (in y-units) into a "width" (in x-units) using the slope constraint $|f(x) - f(x_0)| \leq \ell|x - x_0|$.

Uncertainty Regions After Each Evaluation III

How to Read the Uncertainty Region Formula

The formula says: given that the sawtooth at $x^{(i)}$ is $y^{(i)}$ (our lower bound on f near $x^{(i)}$) and the best value we have found so far is $y_{\min}$, the true global minimum could be in the range $[y^{(i)}, y_{\min}]$. The horizontal width of the region where this improvement could be hiding is $(y_{\min} - y^{(i)})/\ell$. As we sample more, the sawtooth tightens, the dips get smaller, and the uncertainty regions shrink to zero.

Limitations of the Shubert-Piyavskii Method:

1. **Requires knowing ℓ.** In practice the Lipschitz constant is often unknown. If ℓ is too small, the lower bound is invalid (the true function may lie below the sawtooth). If ℓ is too large, the lower bound is very loose and convergence is extremely slow. Estimating a good ℓ is itself a non-trivial problem.
2. **One dimension only.** The method works on an interval $[a, b]$ in one dimension. Extending to multiple dimensions requires specialised algorithms such as DIRECT (Dividing Rectangles) or LIPO, and the number of evaluations needed grows exponentially with the number of dimensions (the "curse of dimensionality").

Algorithm 3.5: Shubert-Piyavskii in Python

```python
import numpy as np

def shubert_piyavskii(f, a, b, ell, eps=1e-5, max_iter=500):
    """Global minimum of f on [a, b] via sawtooth lower bound.

    ell      : Lipschitz constant (upper bound on |f'(x)| everywhere)
    eps      : optimality-gap tolerance: stop when gap <= eps
    max_iter : safety limit on iterations
    Returns (x_best, f_best) -- best point found.
    """
    x1 = (a + b) / 2.0           # start at midpoint
    pts = [(x1, f(x1))]          # list of (x, y) sample pairs
    delta = np.inf                # current optimality gap

    for _ in range(max_iter):
        if delta <= eps:
            break

        best = {'i': 0, 'x': 0.0, 'y': np.inf}

        # Check left boundary intersection with first V-shape
        y_left = pts[0][1] - ell * (pts[0][0] - a)
        if y_left < best['y']:
            best = {'i': 0, 'x': a, 'y': y_left}

        # Check right boundary intersection with last V-shape
        y_right = pts[-1][1] - ell * (b - pts[-1][0])
        if y_right < best['y']:
            best = {'i': len(pts), 'x': b, 'y': y_right}

        # Check all interior sawtooth valleys between adjacent samples
        for i in range(1, len(pts)):
            A, B = pts[i-1], pts[i]
            t = ((A[1] - B[1]) - ell * (A[0] - B[0])) / (2 * ell)
            P = (A[0] + t, A[1] - t * ell)    # valley (x, y)
            if P[1] < best['y']:
                best = {'i': i, 'x': P[0], 'y': P[1]}

        delta = f(best['x']) - best['y']
        pts.insert(best['i'], (best['x'], f(best['x'])))

    return min(pts, key=lambda p: p[1])
```

Shubert-Piyavskii: Code Walkthrough I

Let us trace through the key steps of the implementation to understand what each part does and why.
Initialisation: We sample the midpoint $x_1 = (a + b)/2$ and store it as the first entry in `pts` (a list of (x, y) pairs, sorted by x). The variable `delta` (δ, delta, meaning the optimality gap) starts at infinity because we have no gap estimate yet.
Finding the sawtooth valley: The main loop scans all vertices of the sawtooth to find the lowest one. There are three types of vertices to check:

1. The left boundary a: where the V-shape from the first sampled point intersects the left wall.
2. The right boundary b: where the V-shape from the last sampled point intersects the right wall.
3. Interior valleys: for each pair of adjacent samples A and B, the valley between their V-shapes occurs at the horizontal position where the right-going arm of A's V equals the left-going arm of B's V. Solving for this intersection:

$$t = \frac{(A_y - B_y) - \ell(A_x - B_x)}{2\ell}, \quad P_x = A_x + t, \quad P_y = A_y - \ell \cdot t$$

where A_x, A_y are the x and y coordinates of sample A.

Shubert-Piyavskii: Code Walkthrough II

Sampling: We evaluate f at the lowest sawtooth vertex (the most optimistic location for the global minimum) and insert it into `pts`, keeping the list sorted by x.
Optimality gap update: After evaluating f at the best sawtooth vertex with value `best['y']`, the gap is $\delta = f(x_{\text{new}}) - \texttt{best['y']}$. When $\delta \leq \varepsilon$, the true global minimum cannot be more than ε below the best value found.

3.7 Bisection: Using Derivatives to Find Minima I

All the methods so far use only *function values* $f(x)$. If we also have access to the **derivative** $f'(x)$ (read "f prime of x", the instantaneous slope or rate of change of f at the point x), we can use a qualitatively different and often faster approach.

The key observation is that at any local minimum x^*, the derivative must equal zero: $f'(x^*) = 0$. This is because the function is neither increasing nor decreasing at its minimum. So finding the minimum of f is equivalent to finding a **root** (a zero) of f'.

The **bisection method** solves this root-finding problem using a simple strategy: if the derivative is negative at one end of an interval and positive at the other, then by the *Intermediate Value Theorem* (a fundamental theorem of calculus stating that a continuous function must pass through every intermediate value), there must be a zero of f' somewhere inside the interval. We then halve the interval and repeat.

The Intermediate Value Theorem Applied to f'

If f' is **continuous** (no sudden jumps or breaks) on $[a, b]$, and if $f'(a)$ and $f'(b)$ have **opposite signs** — one is strictly positive and one is strictly negative — then there exists at least one point $x^* \in (a, b)$ (strictly inside the interval) where $f'(x^*) = 0$.

In plain English: if the slope of f is downward (negative f') at the left end and upward (positive f') at the right end, the slope must pass through zero somewhere in the middle. That zero-crossing is exactly the location of a minimum.

The Bisection Algorithm Applied to f'

We apply bisection to the function f' (the derivative), treating zero-crossing of f' as the root to be found.

1. **Check validity:** Verify $f'(a) \cdot f'(b) \leq 0$ (the product of two values of opposite sign is negative).
2. **Evaluate midpoint:** Compute $m = (a + b)/2$ (m, the midpoint, halfway between a and b) and $y_m = f'(m)$.
3. **If** $y_m = 0$**:** We found the root exactly at m; stop.
4. **If sign**$(y_m) =$ **sign**$(f'(a))$**:** (they have the same sign, meaning the zero is in the right half): set $a \leftarrow m$ (discard the left half).
5. **Otherwise:** set $b \leftarrow m$ (discard the right half).
6. **Repeat** until $b - a < \varepsilon$ (epsilon, the interval width tolerance).

Convergence Rate of Bisection

Each iteration *halves* the interval, regardless of the function shape. Starting from an interval of length $|b-a|$, after n iterations the interval has length $|b-a|\,/\,2^n$ (the original length divided by 2 to the power n). To achieve precision ε (epsilon, the target interval length):

$$n = \left\lceil \log_2\left(\frac{|b-a|}{\varepsilon}\right)\right\rceil \quad \text{iterations are needed.}$$

3.7 Bisection: Using Derivatives to Find Minima V

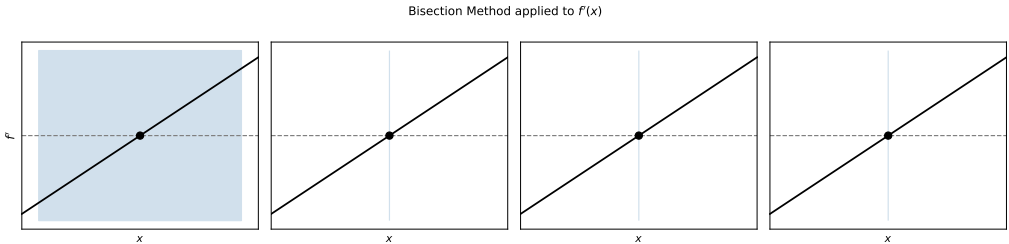
Notation for the Convergence Formula

2^n (two to the power n): the total shrinkage factor after n halvings. After 10 halvings, the interval is $2^{10} = 1024$ times shorter.
$|b - a|$ (absolute value of b minus a): the initial interval length, always a positive number.
$\log_2(\cdot)$ (logarithm base two, read "log base two of"): counts how many times we need to halve to reach the target precision. Specifically, $\log_2(N)$ is the number n such that $2^n = N$.
$\lceil \cdot \rceil$ (the ceiling function): rounds up to the nearest integer, since we need a whole number of iterations.
ε (epsilon): the desired final interval width.

How to Read the Convergence Formula: One Bit Per Step

Each iteration of bisection produces exactly one additional correct *bit* of the answer: it halves the uncertainty interval. Ten iterations give $2^{10} = 1024$ times better precision. Thirty iterations give $2^{30} \approx 10^9$ times better precision — more than a billion-fold improvement. This linear convergence rate is moderate (quadratic fit search is faster near the minimum), but it is *completely reliable*: it works regardless of the function shape as long as f' is continuous and brackets the zero.

Bisection: Four Iterations Visualized I



The figure shows the bisection method applied to the derivative $f'(x)$ across four successive iterations. Reading the panels from left to right:
In each panel, the horizontal axis shows the input x and the vertical axis shows the derivative value $f'(x)$. The horizontal dashed line marks $f'(x) = 0$, the target root. The blue shaded region on the x-axis marks the current bracket. The filled dot shows the midpoint evaluated at this iteration.

Bisection: Four Iterations Visualized II

After evaluating $f'(m)$ at the midpoint m, we observe whether the dot is above or below the dashed line. If it is on the same side as the left endpoint's dot, we discard the left half (set $a \leftarrow m$). If it is on the opposite side, we discard the right half (set $b \leftarrow m$). Either way, the blue shaded region halves in width. By panel 4, the bracket has shrunk to $1/2^4 = 1/16$ of its original width, visibly closing in on the zero crossing.

Algorithm 3.6: Bisection Method in Python

```python
import numpy as np

def bisection(f_prime, a, b, eps=1e-6):
    """Find a root of f_prime on [a, b] using bisection.

    f_prime : callable -- the derivative of the objective function
    a, b    : bracket endpoints; must satisfy f_prime(a)*f_prime(b) <= 0
    eps     : interval width tolerance (default 1e-6)
    Returns (a, b) enclosing the root, with b - a <= eps.
    """
    if a > b:
        a, b = b, a           # ensure a < b regardless of user input

    ya, yb = f_prime(a), f_prime(b)

    if ya == 0:               # root is exactly at left endpoint
        return a, a
    if yb == 0:               # root is exactly at right endpoint
        return b, b

    sign_ya = np.sign(ya)     # record sign of left endpoint (+1 or -1)

    while b - a > eps:
        m  = (a + b) / 2.0    # midpoint
        ym = f_prime(m)       # evaluate derivative at midpoint
        if ym == 0:
            return m, m       # exact root found
        elif np.sign(ym) == sign_ya:
```

Algorithm 3.7: Finding a Sign-Change Bracket

```python
import numpy as np

def bracket_sign_change(f_prime, a, b, k=2.0):
    """Expand [a, b] until f_prime(a) and f_prime(b) have opposite signs.

    k : expansion factor (each step, half-width grows by factor k)
    Returns (a, b) with f_prime(a) * f_prime(b) <= 0, suitable for bisection.
    """
    if a > b:
        a, b = b, a              # ensure a < b

    center     = (a + b) / 2.0
    half_width = (b - a) / 2.0

    while f_prime(a) * f_prime(b) > 0:
        half_width *= k          # expand the half-width by factor k
        a = center - half_width
        b = center + half_width

    return a, b
```

Listing 7: symmetrically until f_prime has a sign change]Python: Expand [a, b] symmetrically until f_prime has a sign change

This utility function prepares a starting bracket for bisection.

- `center`: the midpoint of the initial $[a, b]$, kept fixed.

Bisection: A Worked Numerical Example I

Let us apply bisection to a concrete function and trace every iteration in detail.

Problem

Minimise $f(x) = \frac{1}{2}x^2 - x$ (one-half x squared minus x). We apply bisection to the derivative $f'(x) = x - 1$. Initial bracket: $[a, b] = [0, 1000]$.

Verifying the bracket is valid:

$$f'(0) = 0 - 1 = -1 < 0 \quad \text{and} \quad f'(1000) = 1000 - 1 = 999 > 0.$$

Since $f'(a) = -1$ is negative and $f'(b) = 999$ is positive, they have opposite signs. The bracket is valid and the bisection algorithm guarantees it will find the root.

Iteration trace:

Bisection: A Worked Numerical Example II

Step	a	b	Midpoint m	$f'(m) = m - 1$	Action
0	0	1000	500	$499 > 0$	Set $b \leftarrow 500$
1	0	500	250	$249 > 0$	Set $b \leftarrow 250$
2	0	250	125	$124 > 0$	Set $b \leftarrow 125$
3	0	125	62.5	$61.5 > 0$	Set $b \leftarrow 62.5$
4	0	62.5	31.25	$30.25 > 0$	Set $b \leftarrow 31.25$
5	0	31.25	15.625	$14.625 > 0$	Set $b \leftarrow 15.625$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
≈ 30	≈ 1	≈ 1	≈ 1	≈ 0	Converged

The true minimum is at $x^* = 1$ (since $f'(1) = 1 - 1 = 0$). The number of iterations to reach tolerance $\varepsilon = 10^{-6}$:

$$n = \left\lceil \log_2 \left(\frac{1000}{10^{-6}} \right) \right\rceil = \lceil \log_2(10^9) \rceil = \lceil 29.9 \rceil = 30.$$

Bisection: A Worked Numerical Example III

Interpretation

Exactly 30 iterations are needed, regardless of the shape of f or the location of the minimum. Every iteration halves the interval; after 30 halvings, the original length of 1000 has been reduced to $1000/2^{30} \approx 9.3 \times 10^{-7} < 10^{-6}$. This guaranteed convergence in a predictable number of steps is the hallmark of bisection.

Brent-Dekker: Bisection Plus Super-Charged Speed I

Pure bisection converges at a fixed linear rate (one correct bit per iteration). For many smooth functions we can do much better near the root by fitting a low-degree polynomial through recent points and jumping to its zero. However, these faster methods can occasionally produce points *outside* the current bracket, destroying the convergence guarantee.

The **Brent-Dekker method** (named after Richard Brent and Theodorus Dekker, who developed it independently in the early 1970s) resolves this tension by combining three methods:

1. **Bisection:** Always converges at linear rate $O(2^{-n})$ (order 2 to the $-n$, meaning the error halves each step). Used as a "safety net" whenever the faster methods are unsafe.
2. **Secant method:** Fits a *line* (a polynomial of degree one) through the two most recent points and finds where that line crosses zero. Converges super-linearly at rate $\approx \varphi \approx 1.618$ (digits per digit) but can diverge or jump outside the bracket.
3. **Inverse quadratic interpolation (IQI):** Fits a *parabola* (a polynomial of degree two) through three recent points, but in the *inverse* direction (treating x as the function and $y = f'(x)$ as the variable, then evaluating at $y = 0$). Even faster than the secant method but can also diverge.

Brent-Dekker: Bisection Plus Super-Charged Speed II

At each step, Brent-Dekker uses IQI or the secant method if the predicted point falls inside the bracket and is sufficiently far from both endpoints. Otherwise it reverts to bisection. The result is an algorithm that combines the *reliability* of bisection with the *speed* of quadratic methods.

Brent-Dekker: Bisection Plus Super-Charged Speed III

Using Brent-Dekker in Python (via SciPy)

The standard Python scientific library `scipy` (Scientific Python) provides Brent-Dekker in two forms:
Root-finding form (for when you want to find a zero of f'):

```
scipy.optimize.brentq(f_prime, a, b)
```

This finds a root of `f_prime` on the interval `[a, b]`, requiring a sign change in `f_prime` across the interval.
Minimisation form (for when you want to minimise f directly):

```
scipy.optimize.minimize_scalar(f, bounds=(a, b), method='bounded')
```

This minimises the scalar function f on $[a, b]$ using Brent's hybrid of golden section and parabolic interpolation. The result object's attribute `res.x` holds the minimiser and `res.fun` holds the minimum function value.

Brent-Dekker: Bisection Plus Super-Charged Speed IV

Key Insight: The Practical Standard

For any one-dimensional bracketing problem in practice, prefer `scipy.optimize.minimize_scalar` with `method='bounded'`. It provides the guaranteed convergence of bisection (it *always* converges), the speed of quadratic interpolation (typically super-linear near the minimum), and requires no tuning of parameters. It is the method used inside most professional scientific computing software.

3.8 Summary of Chapter 3 I

This chapter introduced a hierarchy of methods for finding a minimum of a univariate (one-dimensional) function. Each method makes progressively stronger assumptions about f to achieve faster or more powerful convergence. The methods form a toolkit: we choose the appropriate tool based on what we know about the function and what resources (evaluations, derivative access, Lipschitz constant knowledge) we have.

3.8 Summary of Chapter 3 II

All Key Methods at a Glance

`bracket_minimum`**:** Finds an initial bracketing triple (a, b, c) by stepping from x_0 and doubling the step size each time. Requires only the existence of a local minimum. This is always the *first* step before applying any refinement method.

Fibonacci search: Given a fixed budget of n evaluations on a unimodal function, this is the provably optimal strategy — no other algorithm can guarantee a smaller final interval. The bracket shrinks by the exact Fibonacci factor F_{n+1}. Must know n in advance.

Golden section search: Uses the constant placement ratio $\rho = \varphi^{-1} \approx 0.618$ at every step. Slightly suboptimal relative to Fibonacci for small n, but effectively identical for $n > 10$. Can be run as an anytime algorithm.

Quadratic fit search: Fits a parabola to three bracketing points and jumps to its vertex. Achieves super-linear convergence near a smooth minimum. Requires smoothness; safeguards needed for degenerate cases.

Shubert-Piyavskii: A global optimisation method that builds a sawtooth lower bound using the Lipschitz constant ℓ (ell). Guaranteed to find the global minimum on $[a, b]$ even for multimodal functions, provided ℓ is known.

Algorithm Comparison Table I

The table below summarises the key properties of all methods in this chapter. Each row corresponds to one method; the columns capture the most important practical attributes for choosing between them.

Algorithm	Needs f'?	Global?	Convergence	Budget
`bracket_minimum`	No	No	— (setup only)	Varies
Fibonacci search	No	No	$O(F_{n+1}^{-1})$	Fixed n
Golden section	No	No	$O(\varphi^{-(n-1)})$	Any
Quadratic fit	No	No	Super-linear	Any
Shubert-Piyavskii	No	**Yes**	Depends on ℓ	Adaptive
Bisection	**Yes**	No	$O(2^{-n})$	Fixed ε
Brent-Dekker	**Yes**	No	Super-linear	Adaptive

Column glossary:

- *Needs f'?* — Does the method require evaluating the first derivative (slope) of f? Methods that do not are usable for black-box functions where derivatives are unavailable.

Algorithm Comparison Table II

- *Global?* — Does the method guarantee finding the global minimum of a potentially multimodal function?
- *Convergence* — The rate at which the error decreases. $O(2^{-n})$ (order 2 to the $-n$) means the error halves each step (linear convergence); super-linear means faster than any fixed ratio per step; $O(F_{n+1}^{-1})$ means the interval shrinks by the $(n+1)$-th Fibonacci number.
- *Budget* — Whether the evaluation count must be fixed in advance (Fixed n), set as a precision target (Fixed ε), or can be determined adaptively (Any / Adaptive).

Selected Exercises with Full Solutions I

These exercises test and deepen your understanding of the methods in this chapter. Work through each exercise on your own first, then check the solution.

Exercise 3.1: When to Prefer Fibonacci Over Bisection

Question: Give a concrete example where Fibonacci search is the correct choice over bisection.

Solution: Use Fibonacci search whenever the **derivative f' is not available**. For example, suppose f is the output of a physical experiment (measuring the fuel efficiency of an engine at different throttle settings), a neural network forward pass (no analytical gradient), or a legacy simulation tool. In these cases, computing $f'(x)$ is impractical. Bisection requires a sign change in f' across the bracket, so it *cannot* be used. Fibonacci search requires only evaluations of f itself.

Exercise 3.2: Main Drawback of Shubert-Piyavskii

Question: What is the main practical limitation of the Shubert-Piyavskii method?

Solution: The method requires knowing a valid **Lipschitz constant** ℓ (ell) in advance. In practice, ℓ is often unknown. If ℓ is too small, the sawtooth lower bound may lie *above* the true function at some points, violating the guarantee that the sawtooth is a valid lower bound. If ℓ is too large, the V-shapes are very wide, the sawtooth is very low (a very loose bound), and convergence is extremely slow. There is no automatic way to choose ℓ optimally without additional knowledge of f.

Exercise 3.5: Validity of a Lipschitz Constant

Question: Is $\ell = 2$ a valid Lipschitz constant for $f(x) = (x + 2)^2$ on $[0, 1]$?

Solution: No. The derivative is $f'(x) = 2(x + 2)$. On $[0, 1]$ this ranges from $f'(0) = 2(0 + 2) = 4$ at the left end to $f'(1) = 2(1 + 2) = 6$ at the right end. A valid Lipschitz constant must be an upper bound on $|f'(x)|$ at every point in $[0, 1]$. The maximum is $|f'(1)| = 6$. Since $\ell = 2 < 6$, it is *not* a valid Lipschitz constant for this function on this interval. Choosing $\ell = 2$ would produce a sawtooth lower bound that violates the true function at some points. A safe choice is any $\ell \geq 6$.

Exercise 3.6: Fibonacci Budget for a Given Precision

Question: With 3 evaluations on [1, 32], can we guarantee narrowing the optimum to an interval of length ≤ 10?

Solution: No. With $n = 3$ evaluations, Fibonacci search shrinks the interval by $F_{n+1} = F_4 = 3$. The final interval has length:

$$\frac{b-a}{F_4} = \frac{32-1}{3} = \frac{31}{3} \approx 10.33 > 10.$$

We cannot guarantee an interval of length ≤ 10 with only 3 evaluations. With $n = 4$ evaluations: $F_5 = 5$, giving a guaranteed length of $31/5 = 6.2 < 10$. So 4 evaluations suffice.

Looking Ahead: From One Dimension to Many I

The one-dimensional bracketing methods in this chapter are not just standalone tools — they are the fundamental building blocks that multi-dimensional optimisation algorithms call *repeatedly*.

Chapter 4: Local Descent Methods. When minimising a function of many variables $f(\mathbf{x})$ (bold $\mathbf{x}$ denotes a *vector* of inputs, such as $\mathbf{x} = (x_1, x_2, \ldots, x_d)$), a common strategy is to move in one direction at a time. The amount to move in a chosen direction $\mathbf{d}$ (d bold, a direction vector) is the **step size** α (alpha, pronounced "al-fah", a non-negative real number). Choosing α to minimise $f(\mathbf{x} + \alpha\mathbf{d})$ as a function of the scalar α alone is a one-dimensional minimisation problem — called a **line search** — and golden section or Brent's method is used to solve it.

Line search in gradient descent, conjugate gradient, and quasi-Newton methods. All of these widely-used multi-dimensional methods call a 1D bracketing subroutine at each outer iteration. The efficiency of the line search directly determines the speed of the outer loop.

Shubert-Piyavskii and higher-dimensional global optimisation. The Lipschitz lower bound idea extends to multiple dimensions. The DIRECT algorithm (Dividing Rectangles) and the LIPO method generalise the sawtooth lower bound to boxes in d-dimensional space. However, the number of evaluations needed grows exponentially with the dimension d (the "curse of dimensionality"), so these methods are typically limited to problems with $d \leq 20$ or so.

Looking Ahead: From One Dimension to Many II

The Big Picture

Every multi-dimensional optimiser studied in later chapters has a one-dimensional sub-problem at its heart. The bracketing methods here — the bracket-finding heuristic, golden section search, and Brent-Dekker — will appear again and again as key subroutines. A solid understanding of this chapter is therefore the foundation on which everything else in the course is built.

End of Chapter 3